LaunchPad — Mike-Inspired Capability Upgrades

Audience: Claude Code, working in the LaunchPad (SenseFound) monorepo. **Source of inspiration:** the OSS project `willchen96/mike` (AGPL-3.0). Do **not** copy code verbatim — re-implement patterns in our codebase. **Status:** specification. Not yet ranked against the wider Q3/Q4 roadmap.

---

0. Context for the implementer

LaunchPad is described in full in `BRIEF.md`. The short version: Next.js + Supabase + Anthropic Claude, with a custom agent runtime (`runAgentStream`) that loads personality + memory + tools per project and streams responses with inline `:::artifact{}` blocks. Skills are baked-in workflows that produce typed artifacts. The intelligence layer runs autonomously between sessions.

The features below are inspired by Mike (a legal document assistant). Each is a discrete deliverable, sequenced from highest leverage to lowest. **Do not attempt all four in one PR.** Each epic below should land as its own PR, with its own migration, its own ship-and-release notice for the founder, and updated Acceptance Criteria per the consulting-contract Art. 4-bis.

Before starting any epic: read `BRIEF.md` end-to-end, then read this doc end-to-end, then open the open-questions section (§7) with the team. Do not begin implementation until the open questions for that epic are resolved.

---

1. Sequencing and dependencies

Recommended order, with rationale:

1. **Citation Grammar v1** (§2) — smallest, fastest, highest trust impact. No data-model change required. Touches the agent system prompt, the artifact parser, and the rendering layer. ~3–5 days.
2. **Tabular Reviews** (§3) — new primitive, biggest UX unlock. New tables, new route, new tool family. ~2–3 weeks.
3. **Tracked-Change Edits for Build Artifacts** (§4) — depends on Citation Grammar v1 being shipped (so the edit cards can show source quotes). ~2 weeks.
4. **BYOK + Per-User Model Selection** (§5) — cost / unit-economics play. Mostly settings + key vault. ~1 week.

Hard ordering constraints:

- §4 requires §2 to be GA (the edit cards reuse the citation rendering)
- §3 is independent of all others

- §5 is independent of all others but should land before the M1 verification window (31 Jan 2027) because the per-user €0.25/month cost target depends on it

Soft ordering: §2 unblocks the competitor-comparison and investor-pipeline-analysis features called out in `BRIEF.md` under "What the Platform Should Do," so prioritising it is a multiplier.

---

2. Epic 1 — Citation Grammar v1

Goal

Every factual claim the agent makes about a source (web result, ingested doc, skill artifact, ecosystem alert, intelligence brief) carries a machine-verifiable citation with a verbatim quote bounded to ≤25 words and a stable source ID.

Why now

`BRIEF.md` already states: *"Every factual artifact MUST include a non-empty `sources` array. This is enforced at the prompt level."* In practice, "sources" today is a free-form URL list. There is no grammar, no verbatim quote requirement, and no enforcement beyond a prompt instruction the model can drift away from. Mike's pattern — `[N]` markers in prose + a single `<CITATIONS>` JSON block at end of response — is rigorous, model-tolerant, and post-hoc parseable.

Scope

In scope:

- A citation block grammar (`<CITATIONS>...</CITATIONS>`) emitted after the agent's text response
- An `[N]` inline-marker convention in the agent's prose
- A parser that extracts citations from the streamed response, validates them against the supplied sources, and rejects malformed/unverifiable ones
- Rendering of citation chips inline next to `[N]` markers, with a hover/click panel showing the source quote and a deep link
- Prompt-level enforcement (system prompt section + per-skill prompts)
- Citation persistence on the message (`chat_messages.citations` JSONB column)
- Coverage of three citation source types: `web` (URL + page title + scraped snippet), `skill_artifact` (LaunchPad's own prior skill output), `ecosystem_alert` (a row from `ecosystem_alerts`)

Out of scope (defer to v2):

- PDF page-anchored citations (becomes relevant only when LaunchPad ingests user-uploaded docs — see §6)
- Citation-to-citation deduplication and clustering
- Auto-suggesting alternative sources when a quote can't be verified

Citation grammar

The agent's response stream ends with an optional block:

```
<CITATIONS>
[
  {"ref": 1, "source_type": "web", "source_id": "https://...", "quote": "verbatim text ≤25 words", "title": "page title"},
  {"ref": 2, "source_type": "skill_artifact", "source_id": "research: <artifact_uuid>", "quote": "verbatim text ≤25 words"},
  {"ref": 3, "source_type": "ecosystem_alert", "source_id": "<alert_uuid>", "quote": "verbatim text ≤25 words"}
]
</CITATIONS>
```

Rules (verbatim copy into the system prompt):

- `ref` is a sequential integer (1, 2, 3…) assigned in the order citations appear in prose
- The number inside `[N]` in prose is the `ref` value — NOT a page number, footnote, or any other number
- Every `[N]` in prose MUST have a matching `{"ref": N, ...}` entry
- `quote` must be ≤25 words, verbatim, narrowly scoped to the specific claim
- Don't reuse one quote to support multiple claims — give each its own citation
- Omit the block entirely if there are no citations
- Place the block at the very end of the response, after all prose

Data model

Add a column to `chat_messages`:

```sql
alter table public.chat_messages
  add column if not exists citations jsonb;

create index if not exists idx_chat_messages_citations
  on public.chat_messages using gin (citations);
```

Citation entries persisted as the parsed and validated JSON array. Unvalidated citations (quote not present in source) are stored with a `valid: false` flag rather than discarded, for debugging.

Parser contract

A function `parseCitations(rawResponse: string): { prose: string; citations: Citation[]; errors: ParseError[] }`:

- Extracts the `<CITATIONS>` block, leaves prose unchanged except for the marker positions
- Validates `ref` continuity (1, 2, 3… no gaps, no duplicates)
- For each citation, fetches the source text:
  - `web`: from the cached `web_search` result for that URL within the current turn
  - `skill_artifact`: from the artifact's stored text content
  - `ecosystem_alert`: from `ecosystem_alerts.snippet` or equivalent
- Verifies `quote` substring (case-insensitive, whitespace-tolerant — collapse runs of whitespace before comparing) appears in the source. If not, mark `valid: false`
- Validates `quote.split(/\s+/).length <= 25`
- Returns errors for the agent to retry against (one retry only — don't loop indefinitely)

Agent loop changes

After streaming completes:

1. Run `parseCitations` on the full assistant message
2. If any citations are `valid: false` or there are parse errors, append a system-side correction prompt and ask the agent for one re-emission of the `<CITATIONS>` block only (not the full response). Cap at one retry to avoid cost runaway
3. Persist the validated citations to `chat_messages.citations`
4. Render in the UI

UI requirements

- Inline `[N]` markers in prose are replaced with small numbered chips
- Hovering / tapping a chip opens a popover showing: `quote` in italics, `title` (for web sources) or artifact name, and a "View source" link
- Citations that failed validation render in red with a "Source could not be verified" tooltip and the raw quote — do not hide them, surfacing failures is part of the trust model
- Citation chips reuse existing artifact styling tokens (no new design system primitives)

Prompt changes

Add a `CITATIONS` section to the system prompt (likely in `SOUL.md` or wherever the agent personality and operating rules live — explore the codebase to find this; `BRIEF.md` mentions `SOUL.md` and `AGENTS.md`). The text below is the canonical wording; adapt to the existing prompt's style.

```
CITATIONS:
When you make a factual claim that comes from a web result, a prior skill
artifact, or an ecosystem alert, place a numbered marker [N] inline in your prose
at the point of the claim.

After your complete response, append a <CITATIONS> block containing a JSON array
with one entry per marker:

<CITATIONS>
[
  {"ref": 1, "source_type": "web", "source_id": "https://...", "quote": "verbatim
≤25 words", "title": "page title"}
]
</CITATIONS>

CRITICAL: The number inside [N] in your prose is the "ref" value of an entry in
the CITATIONS block — it is NOT a page number, footnote, or anything else. Refs
are simple sequential integers (1, 2, 3, …) you assign in the order citations
appear.

Rules:
- Only cite text that appears verbatim in the source
- Keep quotes ≤25 words, narrowly scoped to the specific claim
- Don't reuse one quote across multiple different claims — each claim gets its own
citation
- Place the block at the very end of the response. Omit it entirely if there are
no citations
```

Also update the skill prompts (research, simulation, risk-scoring, intelligence-correlator, monitor runs) to reference this convention. The current "every factual artifact MUST include a non-empty sources array" line should be replaced.

Acceptance criteria

1. A user asks "what did our competitor X launch last week?" with web_search enabled. The response contains `[1]` and `[2]` markers; the `<CITATIONS>` block has two verified entries; the UI renders chips that, on hover, show the verbatim quote
2. A user asks a question answerable from a prior research skill artifact. The agent cites `source_type: "skill_artifact"` and the chip deep-links to that artifact
3. A user asks about a recent ecosystem alert. The citation links to the alert and opens the alert detail
4. An adversarial agent response with a fabricated quote (quote not present in source) results in `valid: false` citation persisted, red chip in UI, and a single auto-retry attempting to re-emit a valid block

5. A response with no factual claims (e.g. "okay, will do") has no `<CITATIONS>` block and no parse errors
6. `chat_messages.citations` populated for every assistant message that emits a block
7. The system prompt change is visible in `llm_usage_logs` (prompt tokens go up by a small fixed amount per turn)

Test approach

- Unit tests on `parseCitations` covering: well-formed block, malformed JSON, missing `ref`, duplicate `ref`, quote >25 words, quote not in source, case-insensitive whitespace-tolerant matching, empty / absent block
- Integration test: one end-to-end conversation per source type, asserting persisted citations
- Manual QA: have the agent generate citations for one project's worth of recent activity, eyeball for quality

Non-goals (do not implement)

- Don't try to enforce "every sentence has a citation." Over-citing is as bad as under-citing. The prompt asks for citations on factual claims about sources, not on inferences, opinions, or framing
- Don't add a separate "sources panel" sidebar — the chip-and-popover pattern is sufficient
- Don't migrate prior assistant messages retroactively

---

3. Epic 2 — Tabular Reviews

Goal

Introduce a structured multi-entity comparison primitive: rows are entities (documents, competitors, investors, risks, startups in a portfolio), columns are typed extraction questions, cells are LLM-extracted answers with citations and an explicit value type.

Why now

`BRIEF.md` calls out two long-term features that this primitive directly enables: founder-to-founder benchmarking and multi-project portfolio view for accelerators. It also makes existing surfaces sharper: competitor profiles, investor pipeline, risk audit deep-dives. Right now LaunchPad does structured comparison as prose in artifacts. Mike's tabular pattern is a much better fit.

Concept

A "tabular review" is a saved query:

- A row source — what populates the rows? (e.g. "all competitors from the knowledge graph", "all risks above severity X", "all investors at stage Y")

- A column set — what gets extracted per row? Each column has a `format` constraint
- Cells — the LLM-extracted answer, validated, persisted with citations

Columns formats supported in v1 (mirroring Mike's set):

format	example	validation
`text`	free prose summary	≤200 words
`number`	`12000`	numeric only
`percentage`	`42%`	matches `/^-?\d+(\.\d+)?%$/`
`monetary_amount`	`€1,234,567.89`	currency symbol + numeric
`currency`	`EUR`	ISO 4217 code
`yes_no`	`Yes` / `No`	one of two literals; citation required
`date`	`12 March 2026`	DD Month YYYY or range with em dash
`tag`	one of a configured set	citation required
`bulleted_list`	markdown bullets	each line starts with `*`

Every formatted cell with `yes_no`, `date`, `tag` MUST emit a citation per Epic 1's grammar. The cell's `reasoning` field carries the citation block.

Scope

In scope:

- New data model (`tabular_reviews`, `tabular_cells`)
- A new tool family for the agent (`create_tabular_review`, `read_table_cells`, `add_review_column`, `add_review_row`, `refresh_cell`)
- A new route `/reviews` (collection) and `/reviews/[id]` (detail) with a spreadsheet-style grid UI
- Three seed row-source types: `competitors`, `investors`, `risks`. (Future: `projects` for org-level views, `startup_updates` for journey reviews.)
- Cell extraction LLM call (Haiku by default — these are cheap structured extractions)
- Cell-level refresh ("re-run this row" / "re-run this column")
- Per-cell citations rendered using the Epic 1 grammar

Out of scope (v2):

- Custom row sources (founder upload of a CSV of competitors)
- Cell-level editing by the founder (read-only output for now)

- Sharing reviews across organizations
- Tabular review chats (Mike has `tabular_review_chats` — defer)
- Cross-row aggregations / formulas

Data model

```sql
create table if not exists public.tabular_reviews (
  id uuid primary key default gen_random_uuid(),
  project_id uuid not null references public.projects(id) on delete cascade,
  user_id text not null,
  title text not null,
  row_source text not null
    check (row_source in ('competitors', 'investors', 'risks')),
  row_filter jsonb,  -- optional filter applied to the row source
  columns_config jsonb not null,  -- [{key, label, format, prompt, tags?}]
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index if not exists idx_tabular_reviews_project
  on public.tabular_reviews(project_id);

create table if not exists public.tabular_cells (
  id uuid primary key default gen_random_uuid(),
  review_id uuid not null references public.tabular_reviews(id) on delete cascade,
  row_entity_type text not null,   -- 'competitor', 'investor', 'risk'
  row_entity_id text not null,     -- foreign id within that entity table
  column_key text not null,
  value_raw text,                  -- the model's raw output
  value_parsed jsonb,              -- typed-and-validated value
  citations jsonb,                 -- Epic 1 citation array
  status text not null default 'pending'
    check (status in ('pending', 'extracting', 'ready', 'failed', 'stale')),
  error text,
  extracted_at timestamptz,
  unique(review_id, row_entity_id, column_key)
);

create index if not exists idx_tabular_cells_review
  on public.tabular_cells(review_id, row_entity_id, column_key);
```

`columns_config` shape:

```json
```

```
[
  {
    "key": "pricing_model",
    "label": "Pricing model",
    "format": "tag",
    "prompt": "What pricing model does this competitor use?",
    "tags": ["freemium", "usage-based", "seat-based", "tiered", "free"]
  },
  {
    "key": "last_funding",
    "label": "Last funding date",
    "format": "date",
    "prompt": "When did this competitor most recently raise capital?"
  }
]
```

Tool surface

Add to the agent's tool set (subject to lazy-loading per `BRIEF.md`'s cost rules — these are write tools, exclude on read-only turns):

- `create_tabular_review(title, row_source, columns_config, row_filter?)` → returns `review_id`. Triggers a background pass to populate cells.
- `add_review_column(review_id, column)` → appends a column, extracts cells for all existing rows
- `add_review_row(review_id, row_entity_id)` → adds a row manually
- `refresh_cell(review_id, row_entity_id, column_key)` → re-extracts a single cell
- `read_table_cells(review_id, col_keys?, row_entity_ids?)` → reads a subset; omit either filter to read all

Read tools (loaded always):

- `list_tabular_reviews(project_id)` → titles and IDs
- `get_tabular_review(review_id)` → review + cells

Extraction loop

For each cell:

1. Load row entity (e.g. `competitor` from `competitor_profiles`)
2. Gather context: profile fields, signals linked to this competitor, recent ecosystem alerts mentioning the entity
3. Issue a Haiku call with: column prompt, format constraint (tells the model the exact output format), context, and the Epic 1 citation instructions
4. Parse the response against the format validator

5. If validation fails, retry once with the validation error in the prompt
6. Persist with `status: 'ready'` or `status: 'failed'` + error

Format-specific prompt suffixes (lifted in spirit from Mike's `formatPromptSuffix`):

- `number` → "Respond with a single number only. No units, no explanation."
- `percentage` → "Respond with a single percentage value (e.g. 42%). No explanation."
- `yes_no` → "Respond with Yes or No only. Include a citation per the citation grammar that points to the exact source language supporting the answer."
- `date` → "Respond with the date only in DD Month YYYY format. If a range, separate dates with an em dash. Include a citation."
- `tag` → "Respond with exactly one of: {tags}. No other text. Include a citation."
- `bulleted_list` → "Respond as a markdown bulleted list only — no prose. Each item on its own line, prefixed with '* '."

UI requirements

`/reviews` index page:

- Linear-style list of reviews with title, row source, column count, cell-completion progress
- "New review" button → modal with row-source selector and column builder
- Reuse `HeaderSearchBtn` / shared toolbar primitives if they exist

`/reviews/[id]` detail page:

- Spreadsheet grid: sticky first column (row entity name), columns from `columns_config`
- Cell states rendered distinctly: pending (skeleton), extracting (spinner), ready (value), failed (red, with retry button), stale (greyed)
- Click a cell → side panel showing the value, citations (Epic 1 chips), source context, and a "re-extract" action
- Click a column header → "edit column" / "refresh column" actions
- Click a row → "open entity" (links to competitor profile, investor detail, or risk detail page)

Cost notes

A review with 20 rows × 5 columns = 100 Haiku calls. At current pricing this is roughly $0.05–$0.15 depending on context size. Treat each cell extraction as a separate `llm_usage_logs` entry, tagged `step: 'tabular_review_extraction'`. Check budget before the bulk fill and skip cells if over budget (set status `pending`, surface as "Budget cap reached — review the budget settings to continue").

Acceptance criteria

1. A founder creates a competitor comparison review with 5 competitors × 4 columns (pricing model: tag, last funding: date, team size: number, has API: yes_no). All 20 cells populate

within 60 seconds. Each `yes_no` and `date` cell has a citation
2. Adding a new column triggers extraction for all existing rows
3. Refreshing a cell re-extracts only that cell and updates the timestamp
4. A cell that fails extraction shows an error and a retry button; retry works
5. Stale cells (>14 days old, mirror the skill staleness rule from `BRIEF.md`) render greyed and are eligible for batch refresh
6. Cells respect the per-project budget cap; over-budget extractions are marked pending with a budget message
7. The detail page renders 100 cells without jank
8. Deleting a review cascades to cells

Test approach

- Unit tests on each format validator (numeric, percentage, monetary, currency, yes/no, date, tag, bulleted_list)
- Integration tests: full review lifecycle (create → populate → add column → refresh cell → delete)
- Manual QA with the three seed row sources

Non-goals

- Don't build a formula bar / aggregation engine
- Don't try to make this realtime-collaborative
- Don't expose this as a chat-driven feature in v1 (founders go to the page directly). v2 can add `create_tabular_review` from chat

---

4. Epic 3 — Tracked-Change Edits for Build Artifacts

Goal

Allow the agent to propose precise text edits to the artifacts it has already generated (pitch deck, one-pager, landing page copy), surfaced as Accept/Reject cards rather than wholesale regenerations.

Why now

Today the build skills regenerate the whole artifact on every iteration. Founders iterate ("change the headline, make slide 4 punchier, swap the TAM number") and lose every prior edit. Mike's tracked-change pattern is the right primitive: minimal `find`/`replace` substitutions anchored by ~40 chars of `context_before` / `context_after`, rendered as approval cards.

Dependency on Epic 1

Each edit card should display the agent's reason and, if the edit is sourced from a citation (e.g.

"Series A landed at $12M per Crunchbase"), the Epic 1 citation popover. Without Epic 1 in place, edit cards lose their source-grounding story.

Scope

In scope:

- A new tool `propose_artifact_edits(artifact_id, edits[])` where each edit is `{find, replace, context_before, context_after, reason}`
- A diff engine that locates and applies edits to the artifact's text representation. Markdown for landing pages and one-pagers, structured JSON for pitch decks (each slide has text fields)
- An `artifact_edits` table tracking pending / accepted / rejected state per edit
- A versioning chain for build artifacts (`build_artifact_versions`)
- UI: per-edit Accept / Reject cards in the chat stream, plus a side-by-side diff view on the artifact detail page

Out of scope (v2):

- DOCX tracked-change emission (Mike's `<w:ins>` / `<w:del>` machinery). Defer until founders ask for editable Word output
- Multi-author tracked changes
- Conflict resolution between overlapping edits — for v1, second-edit-wins-after-first-applied; document this clearly

Data model

```sql
```

```sql
create table if not exists public.build_artifact_versions (
  id uuid primary key default gen_random_uuid(),
  artifact_id uuid not null references public.build_artifacts(id) on delete cascade,
  version_number integer not null,
  content_md text,           -- for markdown artifacts
  content_json jsonb,        -- for structured artifacts (pitch deck)
  source text not null check (source in (
    'generated', 'user_edit', 'agent_edit', 'user_accept', 'user_reject'
  )),
  created_at timestamptz not null default now(),
  unique(artifact_id, version_number)
);

create table if not exists public.artifact_edits (
  id uuid primary key default gen_random_uuid(),
  artifact_id uuid not null references public.build_artifacts(id) on delete cascade,
  chat_message_id uuid references public.chat_messages(id) on delete set null,
  version_id uuid not null references public.build_artifact_versions(id) on delete c
  edit_id text not null,            -- agent-assigned ID, e.g. "edit-001"
  find_text text not null,
  replace_text text not null default '',
  context_before text,
  context_after text,
  reason text,
  status text not null default 'pending'
    check (status in ('pending', 'accepted', 'rejected')),
  created_at timestamptz not null default now(),
  resolved_at timestamptz
);
```

Edit application algorithm

For markdown artifacts: locate `context_before + find + context_after` in the source. If unique, apply. If not unique, fail the edit with a `reason: "ambiguous"` error returned to the agent. If absent, fail with `"not found"`.

Keep edits as a separate layer until accepted — the persisted artifact text doesn't change until the founder hits Accept. Rejection is permanent; the agent should not re-propose the same exact edit in the same chat session (track in turn state).

For structured artifacts (pitch deck JSON), `find`/`replace` applies within text fields of slides. The edit specifies `slide_id` + the text-field path; the find/replace logic is the same scoped to that field.

Tool surface

```ts
```

```
propose_artifact_edits({
  artifact_id: string,
  edits: Array<{
    edit_id: string,            // agent-assigned, unique within this call
    find: string,
    replace: string,
    context_before?: string,    // ~40 chars
    context_after?: string,
    slide_id?: string,          // for structured artifacts
    field_path?: string,        // for structured artifacts
    reason: string
  }>
}) → { applied: number, errors: Array<{edit_id, reason}>, version_id: string }
```

The tool validates anchors, persists pending edits, returns errors per failed edit. The agent should NOT call this tool more than once per turn. Hard-cap at 20 edits per call.

UI

Inline in chat: a compact card per edit with:

- "Replace «old text snippet» → «new text snippet»" (truncated to ~80 chars each, full text in a tooltip)
- The agent's `reason`
- Accept / Reject buttons
- Citation popover if the edit's reason references a citation from this turn's `<CITATIONS>` block

On the artifact detail page: a "Pending edits" bar at top showing N pending, with "Review all" → side-by-side diff view, Accept-all / Reject-all actions.

Acceptance criteria

1. The agent generates a pitch deck. The founder asks "make the TAM slide stronger and update the headline." The agent proposes 3 edits via `propose_artifact_edits`. The chat renders 3 accept/reject cards. Accepting all produces version 2 of the artifact
2. Rejecting an edit leaves the artifact unchanged; rejected edits persist with `status: rejected` for audit
3. An ambiguous anchor returns an error to the agent and is not surfaced as a card
4. The artifact detail page shows version history with timestamps and source attribution
5. The agent does not regenerate the full artifact when ≤5 edits would suffice (test by prompt evaluation)

Test approach

- Unit tests on the anchor-matching algorithm (unique match, ambiguous, not-found, whitespace tolerance)
- Integration: full edit lifecycle for one markdown artifact and one structured artifact
- Adversarial: agent proposes overlapping edits → reject the second one with a clear error

Non-goals

- Don't try to write `<w:ins>` / `<w:del>` into a Word file in v1. Mike does this; it's gnarly (1000+ lines in `docxTrackedChanges.ts`) and AGPL-encumbered. Build the in-database tracked-change pattern first; export to Word becomes a separate epic if founders ask
- Don't auto-accept edits, ever. Approval is the model

---

5. Epic 4 — BYOK and Per-User Model Selection

Goal

Let users supply their own Anthropic API key (encrypted at rest) and choose their preferred model per use case, reducing LaunchPad's effective COGS for power users and giving the cohort a route past the default $5/project monthly cap.

Why now

`BRIEF.md` and the KPI fragment together target `<€0.25/user/month` for Layer 1. The autonomous intelligence layer (monitors, weekly correlation, stale-skill refresh) makes that target tight for engaged users. BYOK is the standard answer: heavy users pay their own provider bill; LaunchPad's costs scale with light users only.

Scope

In scope:

- A `user_api_keys` table with encrypted key storage (AES-GCM, iv + auth_tag pattern)
- A `/account/api-keys` page where users can add / remove their Anthropic key
- A `user_settings.preferred_model` column (`auto` / `sonnet` / `haiku`)
- Backend: when the user has a key, route LLM calls through it instead of the system key. Log to `llm_usage_logs` with a `byok: true` flag and `effective_cost: 0` from LaunchPad's perspective (but still record token counts)
- Display in the budget UI: "You're using your own key — these costs don't count against your project cap"

Out of scope (v2):

- Multi-provider support (Gemini, OpenAI). Single-provider is fine for v1

- Org-level shared keys
- Cost dashboards for the user's own provider bill

Data model

```sql
create table if not exists public.user_api_keys (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  provider text not null check (provider in ('anthropic')),
  encrypted_key text not null,
  iv text not null,
  auth_tag text not null,
  last_used_at timestamptz,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique(user_id, provider)
);

alter table public.user_profiles
  add column if not exists preferred_model text not null default 'auto'
    check (preferred_model in ('auto', 'sonnet', 'haiku'));
```

Encryption secret in `USER_API_KEYS_ENCRYPTION_SECRET` env var (require ≥32 bytes). AES-256-GCM.

Settings UI

`/account/api-keys`:

- One row per provider with status (configured / not configured)
- Add key: paste field, "Save" button, confirmation toast
- Remove key: confirm dialog, then delete
- "Last used" timestamp displayed for diagnosability

`/account/preferences`:

- Model preference radio: Auto (smart routing, default) / Sonnet (always quality) / Haiku (always cheap)
- Brief explainer next to each

Acceptance criteria

1. A user adds their Anthropic key. The next chat turn uses that key. `llm_usage_logs` records the turn with `byok: true`
2. The user's project budget tile no longer ticks up for BYOK turns

3. Removing the key reverts behavior to the system key
4. Per-model preference is respected: `haiku` routes all turns to Haiku; `sonnet` routes all to Sonnet; `auto` keeps current behavior
5. Encryption round-trip is correct (decrypt-then-encrypt produces the same key)
6. A user cannot read another user's key via any API path

Test approach

- Unit tests on the encryption helpers (encrypt → decrypt round trip; failed auth_tag throws)
- Integration: BYOK key configured, chat turn flows through the user's key (mock the Anthropic SDK)
- Security review: confirm key never appears in logs, error messages, or LLM context

Non-goals

- Don't expose other providers in v1. Mike supports Gemini/OpenAI; LaunchPad's prompts are Anthropic-tuned. Multi-provider is a much bigger lift
- Don't try to validate the key by making a real API call on save (rate-limit / cost concerns). Mark as "unverified" and let the first real chat turn surface a failure

---

6. Cross-cutting requirements (apply to all epics)

- **Telemetry:** every new LLM call writes to `llm_usage_logs` with a precise `step` value (e.g. `tabular_review_extraction`, `citation_retry`, `artifact_edit_proposal`). Keep the existing `project_id` + `user_id` tagging
- **Locale:** every UI string supports `en` and `it` per `BRIEF.md`. Add Italian strings to whatever localization mechanism exists
- **Memory events:** record significant operations in `memory_events`:
  - `citation_verified` / `citation_failed` (one per turn, summary)
  - `tabular_review_created` / `tabular_review_refreshed`
  - `artifact_edits_proposed` / `artifact_edits_resolved`
  - `byok_key_added` / `byok_key_removed`
- **Auth and access control:** reuse the existing project-scoped access pattern. All new tables get RLS policies matching the project access model
- **No AGPL contamination:** do not copy code from `willchen96/mike` (AGPL-3.0). Read for inspiration; re-implement against LaunchPad's stack. In particular, the `docxTrackedChanges.ts` file is the highest-risk paste target — do not look at it as a reference, even
- **PR hygiene:** each epic is one PR. Each PR includes: migration, code, tests, locale strings, a short `RELEASE_NOTES_<epic>.md`, and a screenshot of the UI change. Mark the

corresponding Acceptance Criteria from this doc as ✅ / ⚠️ / ❌ in the PR description

---

7. Open questions to resolve before implementation

Block on these before starting the relevant epic.

Epic 1 — Citations

- Where does the agent system prompt live exactly? `BRIEF.md` mentions `SOUL.md` and `AGENTS.md` and `pi-agent.ts`. Confirm the file path and how locales are handled (`.it.md` variants)
- For `skill_artifact` citations, what is the canonical artifact ID format? `BRIEF.md` mentions `:::artifact{}` blocks parsed by `artifact-parser.ts` — confirm artifacts get stable UUIDs
- Should the citation retry happen silently or be surfaced to the founder? Recommendation: silent on success, visible on failure (red chip + tooltip)

Epic 2 — Tabular Reviews

- What's the current entity model for competitors? `BRIEF.md` mentions `competitor_profiles` — confirm schema and whether risks have a single canonical table (the brief implies they're part of the risk audit but doesn't name the table)
- Should reviews appear in the nav rail (4 primary + 3 secondary routes per `BRIEF.md`) or under a different surface? Recommendation: secondary nav, between Signals and Inbox
- Is Haiku the right default for cell extraction, or should the user's `preferred_model` setting (from Epic 4) win?

Epic 3 — Tracked-Change Edits

- What's the storage format for the pitch deck artifact today? JSON of slides? Markdown? The diff strategy depends on this
- Do landing-page edits need to invalidate the cached deployed page? If so, how is the deploy hook wired?
- Should there be a "regenerate from scratch" escape hatch alongside the edit flow? Recommendation: yes, but make it a confirmed action ("This will discard your manual edits — continue?")

Epic 4 — BYOK

- Does the existing budget UI need a redesign to accommodate the "BYOK doesn't count" state, or is a simple banner sufficient? Recommendation: banner, no redesign in v1
- What happens to a running heartbeat if a user removes their key mid-cycle? Recommendation: fall back to the system key for that cycle, surface a notice

---

8. Anti-goals (do not do)

- Do not import Mike code, even fragments. AGPL-3.0 contamination would force LaunchPad to publish source under AGPL. Re-implement everything against our stack
- Do not bundle these epics into a single mega-PR. Each ships independently
- Do not add multi-provider model support (Gemini, OpenAI). LaunchPad's prompts are Anthropic-tuned and the cost of validating prompt behavior across providers is high
- Do not attempt to retroactively cite past assistant messages. Citations apply going forward only
- Do not auto-accept any agent-proposed change. Everything routes through founder approval, per `BRIEF.md`'s philosophy

---

9. Definition of done (per epic)

An epic is "done" when:

1. All Acceptance Criteria above are checked off in the PR description with evidence (screenshot, test output, or production link)
2. Migration applied to staging and verified
3. `llm_usage_logs` shows the new step values populated
4. Locale strings present for both `en` and `it`
5. Memory events firing for the listed operations
6. PR review approved by at least one team member outside the implementing author
7. Short `RELEASE_NOTES_<epic>.md` added under `docs/release-notes/` summarising the change, the new tool/route surface, and any breaking points

---

10. Future epics (not specified here, pointers only)

These came up in the Mike analysis but did not make this doc. They're worth their own specs later.

- **PDF page-anchored citations** — extending Epic 1 to support `[Page N]` markers when LaunchPad ingests user-uploaded docs (data room files, term sheets)
- **User-authored shareable skills** — Mike's `workflows` + `workflow_shares` + `hidden_workflows` pattern, applied to LaunchPad skills, opens up the accelerator-skills-marketplace vision in `BRIEF.md`
- `find_in_artifact` **tool** — Ctrl+F-style cheap lookup across past skill outputs, to reduce read-full-artifact context costs

- `replicate_project` **action** — Mike has `replicate_document`; the LaunchPad analog is forking a project to A/B-test two pivot directions
- **DOCX export with tracked changes** — Epic 3 stops at the in-database tracked-change model. Real Word export with `<w:ins>` / `<w:del>` is the natural next step if founders demand editable Word output

---